

# Impossibility Results for Post-Compromise Security in Real-World Communication Systems

Cas Cremers  
CISPA Helmholtz Center for  
Information Security  
cremers@cispa.de

Niklas Medinger  
CISPA Helmholtz Center for  
Information Security,  
Saarland University  
niklas.medinger@cispa.de

Aurora Naska  
CISPA Helmholtz Center for  
Information Security,  
Saarland University  
aurora.naska@cispa.de

November 25, 2024 – Version 1.0.1\*

## Abstract

Modern secure communication systems, such as iMessage, WhatsApp, and Signal include intricate mechanisms that aim to achieve very strong security properties. These mechanisms typically involve continuously merging fresh secrets into the keying material that is used to encrypt messages during communications. In the literature, these mechanisms have been proven to achieve forms of Post-Compromise Security (PCS): the ability to provide communication security even if the full state of a party was compromised some time in the past. However, recent work has shown these proofs cannot be transferred to the end-user level, possibly because of usability concerns. This has raised the question of whether end-users can actually obtain PCS or not, and under which conditions.

Here we show and formally prove that communication systems that need to be resilient against certain types of state loss (which can occur in practice) fundamentally cannot achieve full PCS for end-users. Whereas previous work showed that the Signal messenger did not achieve this with its current session-management layer, we isolate the exact conditions that cause this failure, and we show why this cannot be simply solved in communication systems by implementing a different session-management layer or an entirely different protocol. Moreover, we clarify the trade-off of the maximum number of sessions between two users (40 in Signal) in terms of failure-resilience versus security.

Our results have direct consequences for the design of future secure communication systems and could motivate either the simplification of redundant mechanisms or the improvement of session-management designs to provide better security trade-offs with respect to state loss/failure tolerance.

---

\*We provide a list of main changes in Appendix A.

# 1 Introduction

Modern communication systems include intricate mechanisms that aim to achieve very strong security properties. One of these properties is *Post-Compromise Security (PCS)* [16]: Even after the entire state of a device is compromised at some point by an adversary, future communications can still obtain meaningful security properties (esp. against active adversaries). The high-level intuition is that this can be achieved by continuously merging fresh secrets into the keys used for message encryption, in such a way that if the adversary is passive for a short time, it becomes locked out of the communications again, effectively *healing* the communication.

Many globally used communication systems, like Apple iMessage, WhatsApp [43], Signal messenger [37], Google Messages [24], and Element [39] use intricate mechanisms to implement such a continuous merging in of fresh secrets. The canonical mechanism to implement this was introduced by Signal and is called the Double Ratchet [31, 35] (and notably its sub-component, the asymmetric ratchet). In the literature, the Double Ratchet and related PCS mechanisms have been studied extensively, e.g., [3, 9, 11, 12, 15, 17, 18, 25, 26, 29, 35, 38]. The majority of these works prove for some protocol that, if a healing step occurs after a compromise – a key refresh where the adversary is temporarily passive – then security is recovered, thus satisfying PCS.

However, recent work [20] showed that this PCS guarantee does not necessarily lift to the application level. For example, for the Signal messenger, the underlying Double Ratchet can be proven to provide PCS. However, end users do not obtain a similar PCS guarantee, because there is a mismatch between the single stream of messages that users see and the set of up to 40 Double-Ratchet sessions that underlies this stream. In reality, the Signal messenger allows any adversary that has the long-term secrets of A to start a new Double Ratchet session with B, even if A and B previously healed all their ongoing sessions. From a user’s perspective, this means that their single stream of messages can never heal, and they thus do not obtain PCS. The user-level PCS guarantee was coined *conversation-based PCS* in [20], and we refer to it as *conversation PCS*. The PCS failure is not caused by a bug in Signal, but it is a design choice by its designers: For a number of reasons, an end user might lose (parts of) their latest local state. To enable such a user to still communicate with their partners, the Signal messenger allows the creation of a new Double Ratchet session. As a side-effect, an adversary may also exploit this possibility to start a new session without knowing the latest local state, thereby undoing any positive effects from healing.

This has led to the following open question: *Can real-world communication systems actually achieve full conversation PCS, or is this fundamentally impossible, and if so, for what reasons?*

In this work we solve this question and show that communication systems that need to be resilient against certain types of state loss fundamentally cannot achieve full conversation PCS. Our results imply that real-world messengers like Signal and others, which are deployed on hardware platforms where memory might be slightly unreliable (e.g., due to bit flips) or operating-system level backups might be used, cannot hope to achieve the full PCS guarantees.

However, this does not mean that we cannot do better in practice. As already shown in [20], the Signal messenger could obtain better guarantees after compromise by slightly modifying its session-management layer, which determines when and how different (possibly concurrent) Double Ratchet sessions are initiated between two parties A and B. In this work, we go beyond this protocol-specific result and explore several parts of the design space for generic communication systems that aim for PCS-like guarantees. Notably, we show which session-management policies do not contribute to better PCS guarantees and which session handling policy offers stronger PCS guarantees than currently deployed protocols. Moreover, we demystify the effects of the maximum number of concurrent sessions in such protocols, enabling protocol designers to make more informed choices.

**Contributions.** Our main contributions are:

1. We show, and formally prove, that full PCS for an end-user is fundamentally unachievable in communication systems that need to be resilient against certain forms of state loss, which includes all real-world messaging apps.
2. We explore the design space of these communication systems and show how real-world requirements lead to a violation of conversation PCS for both deployed and proposed solutions.
3. We generalize and extend previous Signal-specific findings to arbitrary communication systems and show how to instantiate a session-management layer with a stronger PCS guarantee than what is currently achieved in deployed systems. We clarify the impact of the choice of the parameter that controls the maximum number of concurrent sessions between two users.

We want to highlight that our results also apply to the continuous group key agreement (CGKA) setting, but do not generalize to structurally different approaches for group solution (e.g., sender keys). See Section 3.5 for more details.

We prove our formal statements using the TAMARIN prover. The strength and generality of our proofs comes from our clever choices of abstractions for our formal model of an abstract communication system (see Section 4). All of our formal models, proofs, and results are available in [21] for reproducibility and extensions.

**Outline.** We provide background on the PCS property and communication systems in Section 2. We explore the design space, define the problem and provide intuition on our statements in Section 3. In Section 4, we present our formal models and analysis on the impossibility results. In Section 5, we describe our proposed solution and its formal analysis. We discuss interesting insights and future work in Section 6, and we end with our conclusions in Section 7.

## 2 Background

### 2.1 Post-Compromise Security (PCS)

Post-Compromise Security (PCS) is a guarantee that ensures the security of future communication following the compromise of all secrets of a device. The messages exchanged between the honest parties become confidential again after the so-called "healing", a period during which the attacker remains passive and does not interfere with the honest parties exchanging messages [16]. Typically, the healing serves to refresh their secrets by incorporating untempered entropy, e.g., Diffie-Hellman ephemeral secrets, which removes the attacker from the conversation.

PCS was initially defined in two versions which differ in the capabilities the attacker has: a) Weak-PCS, where the attacker only has access to the long-term secrets during the compromise period, and b) Full-PCS, where the attacker learns the long-term secrets of the victim permanently. In later work [20], PCS was defined in the context of secure messaging in terms of the level of security it achieves: a) session-level PCS, where messages exchanged in a session are secure after the session secrets are compromised, and b) conversation-level PCS, where the messages exchanged in any session of the conversation are secure after a single session is compromised. [13] defines PCS with a single-session specific metric to enable the comparison of the achieved security on different protocols and adversaries. The property has seen an extensive line of research, most prominently in secure messaging [3, 9, 11, 12, 15, 17–19, 25, 26].

## 2.2 Communication Systems

In this section, we give an overview on deployed communication systems, the properties they aim to provide, and how they manage concurrent sessions.

**Underlying Protocols.** The Signal protocol, composed by the X3DH protocol [31] and the Double Ratchet [35], is one of the most widely deployed messaging protocols. It is used by apps like the Signal messenger [37], WhatsApp [43], Google Messages [24], and many more. The protocol is constructed such that after an authenticated key exchange to bootstrap the communication, the users derive a shared symmetric key. From the shared key, they derive the messaging keys using the Double Ratchet [35]. The main idea of the algorithm is to maintain a chain of keys and update it by introducing new entropy—ephemeral Diffie-Hellman keys—every time the users exchange messages going back and forth. The intricate mechanism aims to achieve forward secrecy and post-compromise security [3, 15] for the users.

Matrix [39] is another messaging protocol constructed by two algorithms: Olm and Megolm. Olm [41] is constructed by the Triple Diffie-Hellman protocol [32] and the Double Ratchet [35]. Olm is used to send relevant information for establishing a Megolm channel. Then, to actually encrypt messages and to move the state of the users forward, the protocol uses the Megolm Ratchet [40]. Users maintain two uni-directional channels (effectively sessions), one for sending and one for receiving. The protocol provides weak forward secrecy, and it aims to achieve some version of PCS by rotating its sessions.

Other protocols include Apples’s iMessage PQ3 protocol, IETF’s Messaging Layer Security (MLS) protocol [7], and Wire’s Proteus Protocol [44], which is an implementation of the Signal protocol.

**Properties.** Some basic security guarantees of communication protocols include *authentication*, *confidentiality*, and *integrity*. Stronger guarantees refer to how the communication protocols react in case of a compromise. *Forward secrecy* states that messages exchanged before a compromise are secure. *Post-compromise security* states that messages exchanged after a compromise are secure after healing. Other properties, related to how the protocol reacts in case of lost or out-of-order messages, are a) *immediate decryption*, where messages are decrypted upon being received and placed in their correct order in the conversation, and b) *message-loss resilience*, where future messages can still be decrypted even if some messages are lost by the network.

**Managing Concurrent Sessions.** Due to their inherently concurrent nature, communication systems need to allow for many concurrent connections between their end-users; even between a single pair of end-users. These connections are typically called *sessions*. In practice, communication systems manage these sessions through a so-called *session-management layer*, which is in charge of the creation, the storage, the updating, and the deletion of them.

A prominent session-management layer is the Sesame algorithm [30], which is deployed for the Signal app. In Sesame, each user maintains information about their potential communication partners in the *UserRecords* and their linked devices in the *DeviceRecords* of their local database. For each one of the *DeviceRecords*, Sesame stores a list of sessions shared between the local device and the partner’s device. This list contains one *active* session, and the remaining sessions are marked *inactive*.

The main idea is that the two devices always communicate using the active session, and when a message is received from an inactive one, the latter is promoted to being the current active session. This means that receiving out-of-order messages makes the parties switch their communication to the old session, in which the message originated, in order to decrypt it. A formal analysis of the management layer paired with the Signal protocol and experiments with the deployed system of the Signal app showed that PCS is violated in this setting [20]. A similar work [42] showed that the session-management layers of Wire, called Proteus Dialogue (which

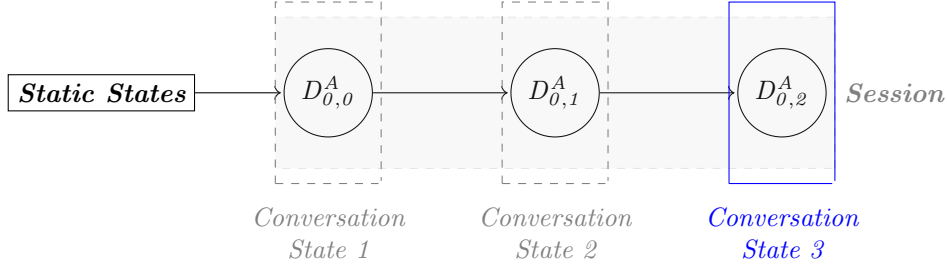


Figure 1: **Evolution of the conversation state of A in the ideal execution.** End-user A bootstraps the conversation using both parties’ static states to derive the dynamic state,  $D_{0,0}^A$ , and subsequently updates it on every message exchange as a long-running session. The ideal environment ensures that both parties can derive and agree on the next dynamic states,  $D_{0,1}^A$ ,  $D_{0,2}^A$ , and so on.

resembles the Sesame algorithm), also suffers from the same vulnerabilities discovered in Signal.

### 3 Resilient Communication Protocols

In this section, we explore the design space of systems that aim to achieve PCS and the trade-off between usability requirements and their implication for security. We define a class of protocols, named *resilient communication protocols*, that abstractly models any two-party protocol that interacts with an unreliable environment. Then, we recall different levels of PCS properties for the end-users and the threat models against which they should hold.

We reason in this abstract class of protocols and draw our main conclusion: For any protocol within this class, i.e., deployed and resilient against real-world failures, the highest level of PCS is fundamentally unachievable. We provide an intuition, through informal arguments and examples, on why our results hold, before setting out to formally prove them in Section 4. We also state our second conclusion on a positive result: Resilient messaging protocols can achieve a stronger PCS guarantee for the end-user than what is currently achieved in practice.

#### 3.1 Abstract Communication System

We consider two end-users, A and B, that wish to communicate with each other using a communication protocol that aims to provide PCS. Each end-user has a single device or client that they use to communicate. For simplicity, we also refer to the devices as A and B. In order to bootstrap the end-to-end communication, each end-user needs a public identifier. In practice, this is typically implemented as a long-term private-public key pair, potentially with a certificate to validate the identity depending on the system used. However, to be as general as possible, we do not assume any specific implementation of the identity but instead refer to the persistent identity as the end-user’s *static state*. We assume that the *static state* is not updated, and we will later return to explaining this choice and the implications of updating or rotating the static state in Section 3.3.

Using the static state, the two end-users authenticate one another and derive secrets to encrypt messages end-to-end. These secrets are often instantiated as symmetric keys, but it could also be an agreement on ephemeral asymmetric keys. Modern communication protocols aim to achieve strong security properties for their users, like post-compromise security, and therefore do not encrypt all messages using the same initial secret. In particular, to achieve PCS, the secrets

need to evolve by incorporating new entropy, such that an attacker can be locked out of the communication if they miss the updates. We define the *dynamic state* of A and B, respectively, to contain all state that changes as messages are exchanged and the secrets to encrypt and decrypt messages between A and B. As the name suggests, the dynamic state is updated with the progression of the communication. A and B can evolve their dynamic states asynchronously, and as messages are exchanged, they should be able agree on the current dynamic state, e.g., A evolves their state to send, and B will evolve their state once they receive the message.

The linear view of the messages exchanged between two end-users is defined as a *conversation*. At any point in time, The *conversation state* contains the snapshot of the underlying dynamic states in this linear view. A communication protocol might have to maintain several concurrent dynamic states, as we will see later, but for now it only needs to contain a single one.

We use the term *ideal execution* to denote an execution of the communication system in the absence of failures, i.e., users can update their state and maintain it synchronized with their partner's. We illustrate such an execution in Figure 1. Next, we provide examples from the real world on how to instantiate the dynamic and the static state.

*Example 1: Static and Dynamic State.* In WhatsApp [43], Google Messages [24], and the Signal app [37], the static state includes the user's identity key pair and the device's identity key pair. The dynamic state is the state of a session: the execution of the X3DH [31] and the subsequent update of the state using the Double Ratchet algorithm [37]. In messengers that adopt the Matrix protocol [39], e.g., Element, the static state includes the device fingerprint and identity key pairs, and the dynamic state is the execution of Olm [41] to establish a session and Megolm [40] to send messages. In Apple's iMessage, the static state includes the long-term user identity key pair, and the dynamic state keeps track of the PQ3 protocol [29, 38] to establish the session and exchange messages.

We refer the reader to Table 1 for a summary of the concepts we presented and will further introduce throughout this section. In the following, we explore components that communication systems consider to enable a seamless conversation, despite any desynchronization or errors that may happen during the communication.

## 3.2 Real-World Failure Modes

In the ideal execution, A and B start a conversation with a shared dynamic state and update it in lockstep—they can always synchronize. However, during the lifespan of a conversation, end-users can become desynchronized. In general, events that disturb or destroy state can be very fine-grained (a single bitflip). To establish general results, we over-approximate all possible types of state loss into two categories: loss of dynamic state and loss of static state. Loss of all state is then modeled as a combination of the two.

**Dynamic state loss** is the event where an end-user loses all the values generated and stored in a dynamic state. The end-user can lose part of the state, but we model the worst case scenario: All stored secrets are no longer accessible in the local memory. Dynamic state loss can happen due to normal operations or can be induced by errors at the hardware layer. We provide two examples below.

*Example 2: Restore Backup.* The end-user restores an old backup in their device, which overwrites the current dynamic state with the dynamic state of the backup. From the device's perspective, a dynamic state loss event took place, and they jumped back to a previous state.

*Example 3: Hardware Failure.* The memory hardware can suffer from failures, e.g., a bit/multi-bit error, or losing part of the state [28]. In the worst case, an unrecoverable error renders the entire dynamic state unusable. This is manifested in the communication system as the inability to encrypt and decrypt messages, amounting to dynamic state loss.

| Layer                  | Concept                       | Definition                                                                                                                                                  |
|------------------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Communication Protocol | Static State                  | The fixed identity of the end-user and their device, typically in the form of public-private key pairs.                                                     |
|                        | Dynamic State                 | The state of a session, which is updated through time or protocol steps by adding new entropy, e.g., the Double Ratchet state.                              |
|                        | Conversation State            | The set of all dynamic states in memory at a particular timepoint.                                                                                          |
| Environment            | Static State Loss             | The event of losing all the memory of an end-user, e.g., because of a lost physical device.                                                                 |
|                        | Dynamic State Loss            | The event of losing or not having the correct dynamic state, e.g., because of hardware failures and restored backups.                                       |
|                        | Dynamic State Compromise      | The event of an attacker compromising and learning the secrets of a dynamic state.                                                                          |
|                        | Static State Compromise       | The event of an attacker compromising and learning the secrets of the static state of a user.                                                               |
| Session-Handling       | Policy                        | Restrictions on how a communication system manages the multiple dynamic states within a conversation, i.e., their creation, storage, deletion and lifespan. |
|                        | Dynamic State Loss Resilience | The ability of the protocol to enable the end-users to recover after losing the dynamic state, to synchronize their state and continue the conversation.    |
|                        | Static State Loss Resilience  | The ability of the protocol to enable the end-users to recover after losing their identity, to synchronize their state and continue the conversation.       |

Table 1: **Summary of concepts.** We categorize the presented concepts according to the communication architecture layer they belong to, namely the communication protocol, the real-world environment, and the session-management layer.

**Static state loss** is the event where an end-user loses their static state and all the dynamic state related to it. The identity can be lost because the user reinstalled the communication system’s client or due to an error in the physical device.

*Example 4: Lost device.* The end-user loses their device, and along with it the static state of the system. In this case, the end-user suffered from a static state loss, and to recover they need to reinstall the client on a new device.

### 3.3 Functional Requirements

As it is possible in the real world for the end-users to lose their dynamic and static state, they can become desynchronized. Let us assume that A loses their state. A and B can no longer communicate with each other since: a) the dynamic state’s secrets are lost for A and they cannot encrypt, b) messages sent by B cannot be decrypted, and c) the app maintains a single dynamic state per conversation, therefore no resynchronization is possible.

If an app were to implement the described protocol, the end-user would be locked out of the conversation every time there is a desynchronization. The app is rendered unusable even during expected operations such as dynamic state loss. The same arguments can be applied to static state loss.

A protocol deployed in the real world should be resilient against events such as dynamic and static state loss. We define this class of protocols as *resilient communication protocols*, as they enable the participants to continue sending and receiving messages after every state loss event. We define the following two properties for the resilient messaging protocol:

**Definition 1.** *Dynamic State Loss Resilience:* After every dynamic state loss, an honest end-user is able to continue the communication, i.e., to send and receive messages.

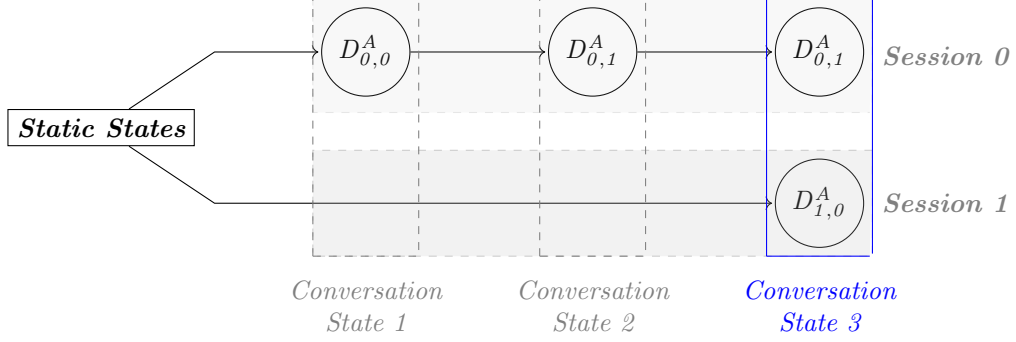


Figure 2: **Evolution of the conversation state of A in the real-world environment.** End-user A starts a new dynamic state  $D_{1,0}^A$  and progresses in the *resilient communication protocol* after the parties are desynchronized and no longer agree on the current dynamic state  $D_{0,1}^A$ .

**Definition 2. Static State Loss Resilience:** After every static state loss, an honest end-user is able to continue the communication, i.e., to send and receive messages.

**Dynamic State Loss Resilience.** Once an end-user loses their dynamic state, the only way to re-synchronize with their partner is to go back to a common point that they both agree on, i.e., the static state. Users know each other’s static state, and from that point they can negotiate new secrets and continue the conversation. In practice, this amounts to A and B executing a new key exchange to authenticate their identities and compute a new shared dynamic state. To achieve this, the conversation allows for multiple dynamic states as shown in Figure 2. In a conversation, A and B have one or multiple running different dynamic states at the same time and overall *multiple* during the lifetime of that conversation.

**Static State Loss Resilience.** After losing the static state, the end-user needs to recover their identity, and re-authenticate, or log-in into their account. The identity could be restored via a local backup, a third party the user has access to, or a recovery mechanism. Re-authentication typically requires an out-of-band channel, for example verifying one’s email address or phone number.

*Rotating the static state.* The static state could be “restored” by rotating the identity, effectively generating new secrets such as long-term identity keys. Notice that in this case, the end-user would depend on another secret or state to authorize the update of the identity, such as a user secret or account password. Since the static state is no longer static, it would be considered part of the dynamic state, and the new secret becomes the static state of the end-user. As a result, all of our statements about the static and dynamic states would simply shift to a different instantiations. I.e., the problem of losing and recovering state would manifest itself at another layer of the secrets. Thus, we fix the static state to be the unique identity of the end-user, which is recoverable using a backup or a recovery mechanism but is not changed otherwise.

### 3.4 Security Requirements

We now define the threat model of the communication system and the different post-compromise security properties.

**Threat Model.** In our work, we consider an attacker that can dynamically compromise any end-user at any time. We define two types of compromise:

1. *Dynamic State Compromise*: The attacker compromises a dynamic state and learns all the stored secrets.
2. *Static State Compromise*: The attacker compromises the static state and learns all the stored identity secrets.

When the attacker compromises both the static and dynamic state, we refer to it as a *full compromise*. Additionally, we consider the attacker to be *active* and interfere into the protocol, e.g., by injecting their own messages with the protocol. For completeness, we will also briefly mention the *passive* attacker, which does not interact with the system but instead records the traffic and tries to decrypt it later (harvest-now-decrypt-later threat model).

**PCS Properties.** From the threat models and the level on which the property should hold, we define four variants of PCS properties for the messaging system.

Initially, we define a PCS variant which is taken from the secure messaging space. Messengers, being an instantiation of a communication system described so far, claim to achieve PCS against a passive eavesdropper.

**Definition 3.** *Session PCS - Passive Attacker*: Security of the session is restored after healing, even when a passive attacker compromises the dynamic state.

Next, we define a PCS variant as it is studied in the literature, which holds over a single session, and where the attacker compromises all user secrets.

**Definition 4.** *Session PCS - Full Compromise*: Security of the session is restored after healing, even when an active attacker compromises the static and dynamic state.

Considering that the conversation of two end-users can have multiple dynamic states, [20] we define the conversation-level variant of PCS against an attacker that can compromise the dynamic state.

**Definition 5.** *Conversation PCS - Dynamic State Compromise*: Security of the conversation is restored after healing, even when an active attacker compromises the dynamic state.

Lastly, we define the conversation-level variant of PCS against the stronger threat model from the literature, where both the dynamic and static state are compromised, i.e., full PCS achieved at the end-user level. The property guarantees that the healing event can lock the attacker out of the entire conversation of the end-user.

**Definition 6.** *Conversation PCS - Full Compromise*: Security of the conversation is restored after healing, even when an active attacker compromises the static and dynamic state, i.e., all the secrets of a conversation.

### 3.5 Overview of Main Results and Consequences

We explored the design space of an abstract communication system: The real-world failures and the functionalities that the app needs to account for, as well as varying levels of PCS guarantees and threat models. We will now provide intuition for our two main results, which we will prove in Section 4 and Section 5.2, respectively.

Our first main result captures a trade-off between resilience and security requirements:

*A resilient communication protocol cannot achieve conversation PCS against a full compromise (Definition 6).*

The underlying intuition is that a resilient protocol tolerates dynamic state loss by allowing users to continue the protocol purely based on their static state. Since this is exactly the state compromised in the full PCS threat model, the attacker can continue communicating by creating a new dynamic state with the partner. Similarly, the protocol tolerates static state loss by allowing the user to recover the *same* static state, making the problem persist across static state recovery. If a protocol has no truly static state, for example because it also rotates long-term keys, it cannot be resilient because after losing dynamic state, it has no shared state to bootstrap from, as discussed previously in Section 3.3.

Our second main result is a positive result:

*A resilient communication protocol can achieve conversation PCS against a dynamic state compromise (Definition 5).*

The intuition is that the resilient protocol allows the users to create new dynamic states (because it tolerates dynamic state loss). Dynamic states can be managed such that state updates ensure that previous dynamic states are deprecated. This way, the protocol can clean the system from a compromised state, and thereby remove the attacker. Thus, if healing occurs after a compromise, the attacker cannot rejoin the protocol, because they do not know either the static state or the current dynamic state.

**Relating our Results to Existing PCS Solutions** At first glance, our results seem to contradict known results from the literature. The difference is that existing work on PCS only considers the single session setting but does not consider resilient communications: When two parties desynchronize, there is no mechanism or security guarantee for any later communications. Indeed, existing solutions do not provide PCS after desynchronization [3, 11, 15, 23, 26, 29, 38]: Their results only hold for single sessions that do not desynchronize and do not carry over to the real-world multi-session deployments.

The authors of [22] propose an authenticated continuous key agreement (ACKA) protocol that can detect and prevent attacks from an active MitM. By treating the Signal protocol as a continuous key agreement, they can provide an add-on to achieve the strong property in the protocol. However, when the protocol is deployed, e.g., in the Signal app, there is no longer a single long-lived running session at the user-level. In reality, parties are allowed to start multiple sessions, and as a result the active MitM attacker is not detected or prevented at the user-level, because of the following behavior: The attacker compromises the identity key, the honest parties heal in a session, the attacker starts a parallel session with the partner.

Our general observations also have implications for group-key-based protocols such as IETF’s Messaging Layer Security (MLS) protocol [7], where desynchronized users can rejoin the group. The rejoining requires the static state of the user. The full PCS property is violated by the obvious attack of the adversary rejoining instead of the compromised party, and since the latter can suffer from dynamic state loss, they cannot tell if they have been impersonated.

Our results also directly apply to the continuous group key agreement (CGKA) setting. They imply impossibility for CGKA with arbitrary group size: If it is already impossible to achieve PCS for groups of size 2, it is also impossible for larger groups. Additionally, our second main result that two-party applications can achieve a stronger PCS property applies to most Signal groups (as they are implemented using peer-to-peer channels). However, different structural approaches for group solutions (e.g., sender keys, which are substantially different from CGKA) each require independent analyses in future work.

In a similar way, work on group messaging [2, 4–6, 10, 14, 36] also considers single session guarantees. The initial work on interoperability of secure messaging in [27] also does not consider PCS for the end user but rather for a single session.

Matrix does not achieve full PCS [1], and the recommendation on how to enable it is to “periodically start a new session” [40] with a “maximum number of at least 4 sessions per device” [39]. Unfortunately, this recommendation leads to worse PCS guarantees because it introduces more opportunities for the adversary to violate PCS. Essentially, the apps from Example 1, including Matrix, allow the following behavior: The static state of A is compromised, A heals, the attacker starts a new session using the compromised state and injects new messages. For the Signal app in particular, this was previously shown experimentally in [20].

## 4 Impossibility Results

In this section, we develop our formal model of a *communication protocol* that aims to achieve PCS with the goal to prove our impossibility result. We do not reason about a specific instantiation (or implementation) of such a protocol, but we create an abstract model that allows us to reason about *any* instantiation that fits into the design space we explored in Section 3. We show the construction of the *resilient communication protocol* model step by step, and we then prove the impossibility result.

First, we explain our methodology in 4.1, and then introduce the tool used for the analysis, the TAMARIN Prover [33] in 4.2. We develop our formal models in 4.3, while highlighting some of the design choices. Finally, we present the results of our analysis in 4.4. We publish our models and results in [21] to enable reproducibility and extensions.

### 4.1 Methodology

Our methodology is to incrementally develop abstract formal models that cover the failure modes, and we show, step by step, the consequences of enabling different behaviors leading to the loss of PCS.

We start with a **Base** model of the communication system defined in Section 3.1. This model captures the basic functions that need to be present in any communication system that aims to provide PCS guarantees. To begin with, we only consider *ideal* executions of the protocol. Building on top of this model, we add the real-world modes of failures, namely a) dynamic state loss, and b) static state loss. In the resulting two non-resilient models, **Non-Resilient<sub>Dynamic</sub>** and **Non-Resilient<sub>Static</sub>**, we prove that indeed communication is stopped after these events: The user is locked out of the protocol.

Motivated by our observation in Section 3.3 that a real-world protocol needs to offer its users a way to recover from state loss, we create the **Resilient<sub>Dynamic</sub>** and **Resilient<sub>Static</sub>** models. In **Resilient<sub>Dynamic</sub>**, we enable multiple dynamic states, which allows the users to continue communication from another dynamic state, and in **Resilient<sub>Static</sub>**, we enable multiple dynamic states and a static recovery mechanism allowing the users to recover from both failures. For both models, we show that they overcome the usability issues of their predecessor model, and we formally prove that they are indeed resilient against the respective failures. However, this resilience comes at a cost: Both models no longer provide PCS (Definition 6), which we showcase via attack traces found in each of the models.

Finally, we use the **Resilient<sub>Static</sub>** model, the most expressive model, to show through attack traces that different classes of policies that manage the multiple dynamic states fail to recover conversation PCS. We are systematic enough to cover different classes of policies:

1. **No policy:** The attacker can compromise old sessions to inject messages [20] and start new sessions.

2. **Sequential sessions:** To mitigate the first attack, creating a new session now deprecates the previous session, making sessions sequential. Deprecated sessions are still kept around to receive messages.
3. **Token-passing between sessions:** Following the logic in [16] of how PCS can be achieved, the active session now passes a token to the next active session when it gets deprecated. This is done to create a chain of sessions, where missed updates remove the attacker from the chain. However, this policy introduces a new point-of-failure: If the active session containing the token is lost, the user is locked out of any further communication. As a result, the protocol is no longer resilient against dynamic state loss.
4. **Third-party providing token:** To improve on the shortcomings of storing the token locally, the protocol can store it with a trusted third party that does not lose state; making this class of protocols resilient against state loss. During protocol execution, the end-users can query the trusted third party with information stored in their static state to retrieve the token when necessary.

## 4.2 The Tamarin Prover

The TAMARIN Prover is an open-source symbolic analysis tool for the analysis and verification of large-scale cryptographic protocols [33]. However, in this work, we use it as a tool to analyze a generic unbounded concurrent system. For instance, we do not make use of its equational theories to model cryptographic primitives or of its inbuilt Dolev-Yao adversary. We provide below a high-level overview of the tool and some of the key concepts on how to model systems and properties.

A user provides the tool with a model of their system, a property they want to verify, and the threat model against which the property should hold. The input of the tool is modeled using a TAMARIN-specific language based on multiset rewriting. In particular, the state of the system is recorded in so-called facts, e.g.,  $\text{Fr}(id_A)$  denotes a special fact, the so-called fresh fact, which guarantess that  $id_A$  is a unique, freshly generated value. To specify how the system can transition from one state to another, the user defines rules. A rule is constructed by a left-hand side (**L**)—the preconditions to execute the transition, a right-hand side (**R**)—the updated system after taking the transition, and the observable logged action (**A**). Rules can also be used to create a threat model by, e.g., modeling the compromise of a party or the attacker executing a protocol step. We provide below an example of a rule that creates a user device. After taking this transition, a new fact  $\text{!UserDevice}(id_A)$  is added to the system whose freshly generated identifier is logged in the action  $\text{!CreateDevice}(id_A)$ . The  $!$  here denotes a persistent fact, i.e., a fact that is never removed from the system even when present on the left-hand side of a rule.

$$[ \text{Fr}(id_A) ] \multimap [ \text{CreateDevice}(id_A) ] \mapsto [ \text{!UserDevice}(id_A) ]$$

To express properties in TAMARIN, the user specifies trace properties using a guarded first-order logic fragment over the observable logged actions, variables, and timepoint variables, which are prefixed with a  $\sharp$ . In the previous example, the observable action is  $\text{CreateDevice}(id_A)$ , logging the creation of the device with identifier  $id_A$ . A simple property over this action could state that there exists a trace in the system where two different devices are created.

$$\begin{aligned} & \exists id_A id_B \sharp i \sharp j . \text{not}(id_A = id_B) \wedge \\ & \text{CreateDevice}(id_A)@ \sharp i \wedge \text{CreateDevice}(id_B)@ \sharp j \end{aligned}$$

To prove a property, TAMARIN negates it and tries to turn the counterexample into a valid trace of the protocol by reasoning backwards, using the user-defined protocol rules and inbuilt

rules for message deduction. If this backwards reasoning fails in *all* cases, TAMARIN concludes that there exists no counterexample, and the property is proven. If a counterexample can be constructed, TAMARIN will output an attack trace. Due to the undecidability of the problem, running out of memory (or time) is unavoidable and always possible. However, in this case, TAMARIN offers a graphical user-interface where the user can inspect partial proofs and manually guide the tool. Additionally, users can create custom heuristics for better proof search, and they can state intermediate lemmas, often called *invariants*, which TAMARIN can use to prove more complex statements.

### 4.3 Modeling the Communication System

In this section, we describe our incremental TAMARIN models of a communication protocol that aims to achieve PCS. We start out with the **Base** model and then explain how each subsequent model builds on the its predecessors. Finally, we model different policies for dynamic state handling in the *resilient communication protocol* model.

#### Base Model

The **Base** model captures the abstract communication system, where two end-users can bootstrap a conversation and update their dynamic state as described in Section 3.1. The model is minimal in the sense that it does not allow for more than a single dynamic state between two users, and every execution of the protocol is *ideal*, i.e., there is no dynamic or static state loss.

**Setup** We model an unbounded number of end-users that communicate with each other in pairs of two. Each of these end-users possesses a single device. Once the parties bootstrap their communication, they can exchange an unbounded number of messages going back and forth.

**Bootstrapping the Protocol** To model the end-users and their static state, we use the persistent fact  $\text{!UserDevice}(id)$ , which carries a unique identifier  $id$ . When two devices start the communication, they create their respective dynamic state from both parties' static state. We show below a simplified rule modeling the creation of a dynamic state.

$$\begin{array}{l}
[ \text{!UserDevice}(id_A), \text{!UserDevice}(id_B), \\
\text{Fr}(s), \text{Fr}(sid) ] \\
\rightarrow [ \text{CreateDynamicState}(sid, id_A, id_B, s) ] \rightarrow \\
[ \text{DynamicStateA}(sid, id_A, id_B, s), \\
\text{DynamicStateB}(sid, id_A, id_B, s), \dots ]
\end{array}$$

Here,  $\text{DynamicStateA}(sid, id_A, id_B, s)$  models  $D_{0,0}^A$ . We use the identifier  $sid$  to refer to the chain of dynamic states, which starts with this one and the value  $s$  to model the secret that is continuously updated as the state evolves. The action  $\text{CreateDynamicState}$  logs the creation of the dynamic state.

**Restricting to Single Session** To allow only a single session of dynamic states between a pair of users, we make use of TAMARIN's restriction feature, and introduce the following restriction:

$$\begin{array}{l}
\forall sid \ sid' \ id_A \ id_B \ s \ s' \ \#i \ \#j . \\
\text{CreateDynamicState}(sid, id_A, id_B, s) @ \#i \\
\wedge \text{CreateDynamicState}(sid', id_A, id_B, s') @ \#j \\
\Rightarrow \#i = \#j
\end{array}$$

This restriction states that if there is a pair of `CreateDynamicState` actions that agree on the users  $id_A$  and  $id_B$ , they are the same one, i.e., only a single dynamic state can be created for a pair of end-users.

**Progressing in the Protocol** Now that the parties have established their conversation, they can act as *Sender* and *Receiver* in an update of their respective dynamic state. We illustrate the rule of A progressing as a sender. The rule for B is analogous:

$$\begin{array}{l}
[ \text{DynamicStateA}(sid, id_A, id_B, s), \text{Fr}(s') ] \\
- [ \text{HonestStep}(sid, id_A, id_B, s), \\
\quad \text{HonestSendStep}(sid, id_A, id_B, s), \dots ] \rightarrow \\
[ \text{DynamicStateA}(sid, id_A, id_B, \langle s, s' \rangle), \\
\quad \text{!UpdateDynStateB}(sid, id_A, id_B, s, \langle s, s' \rangle) ]
\end{array}$$

This rule models A sending a message to B. Notice that we are not using TAMARIN's inbuilt network, but we use the fact `!UpdateDynStateB` to model a message that B can receive. In this rule, A samples new entropy,  $\text{Fr}(s')$  and updates their dynamic state to include it. We use the action `HonestStep` to log the fact that an honest user, in this case A, executed a dynamic state update, and the action `HonestSendStep` to more precisely record that the update was caused by sending a message.

**Threat Model** The attacker can compromise both the dynamic state and the static state of the end-users at any point in time. We use the following facts to model the attacker's knowledge after the compromise: a) `!CompromisedDevice(id)` models the attacker compromising and learning the static state of device  $id$ , and b) `CompromisedDynStateB(sid, id_A, id_B, s)` models the compromise of the dynamic state.

To model the attacker's capabilities, we create *attacker controlled* rules. Below, we showcase the attacker's variant of the rule for being able to receive, decrypt and learn the secret of a dynamic state:

$$\begin{array}{l}
[ \text{CompromisedDynStateB}(sid, id_A, id_B, s), \\
\quad \text{!UpdateDynStateB}(sid, id_A, id_B, s, \langle s, s' \rangle) ] \\
- [ \text{AttackerKnows}(\langle s, s' \rangle) ] \rightarrow \\
[ \text{CompromisedDynStateB}(sid, id_A, id_B, \langle s, s' \rangle) ]
\end{array}$$

We use the action `AttackerKnows` to record that the attacker knows a particular secret. Later on, we will use this to formally state our PCS properties within TAMARIN.

## Modeling State Loss

Now, we model the real-world failure on the communication protocol described in Section 3.1. Starting from the `Base` model, we add dynamic and static state loss, resulting in two new models, `Non-ResilientDynamic` and `Non-ResilientStatic` respectively.

To model dynamic state loss, we add a rule that consumes an existing `DynamicState` fact, which can then no longer be used to progress in the protocol. We show the dynamic state loss of A in the following; the rule for B is analogous.

$$\begin{array}{l}
[ \text{DynamicStateA}(sid, id_A, id_B, s) ] \\
- [ \text{DynamicStateLossA}(\dots) ] \rightarrow \\
[ ]
\end{array}$$

We model the static state loss using similar rules. Additionally, this implies that any previous dynamic states are also unusable. To model the latter, we add a restriction that enforces that after a static state loss no more progress is allowed from any dynamic states of this static state.

$$\begin{aligned}
& \forall \text{sid } id_A \text{ } id_B \text{ } s \text{ } \#i \text{ } \#j . \\
& \text{StaticStateLoss}(id_A)@i \\
& \wedge \text{HonestStep}(\text{sid}, id_A, id_B, s)@j \\
& \wedge \#i < \#j \Rightarrow \perp
\end{aligned}$$

### Recovering From State Loss

To adapt to non-ideal executions of the protocol, we add mechanisms to recover from dynamic state loss and static state loss to the previous model and create the  $\text{Resilient}_{\text{Dynamic}}$  and  $\text{Resilient}_{\text{Static}}$  models.

In the  $\text{Resilient}_{\text{Dynamic}}$  model, we recover from dynamic state loss by removing the restriction that limits dynamic state creation per user pair. This allows for an *unbounded* number of sessions within a conversation. With more than one session between users, communication can now continue in case of a dynamic state loss by simply creating a new dynamic state for further communication.

In the  $\text{Resilient}_{\text{Static}}$  model, we recover from static state loss by adding a new rule that abstractly models an event of recovering the same static state from a trusted third party or backup. The rule raises the  $\text{StaticStateRecovery}(id)$  action. Then, we modify the state loss restriction to allow the parties to progress in the protocol only if they have recovered from the loss:

$$\begin{aligned}
& \forall \text{sid } id_A \text{ } id_B \text{ } s \text{ } \#i \text{ } \#j . \#i < \#j \wedge \\
& \text{StaticStateLoss}(id_A)@i \\
& \wedge \text{HonestStep}(\text{sid}, id_A, id_B, s)@j \\
& \Rightarrow (\exists \#k . \text{StaticStateRecovery}(id_A)@k \\
& \wedge \#i < \#k \wedge \#k < \#j)
\end{aligned}$$

### Policies

We will now describe how we model the family of policies we have identified in Section 4.1.

**No Policy** In the absence of any policy, our  $\text{Resilient}_{\text{Static}}$  suffices since it does not put any restrictions on the dynamic state management.

**Sequential Sessions** The *sequential sessions* policy enforces that only the latest session can encrypt and send new messages, while older sessions are only allowed to receive messages that are still in flight. To implement the first part of the policy, we add the following restriction to our  $\text{Resilient}_{\text{Static}}$  model:

$$\begin{aligned}
& \forall \text{sid } \text{sid}' \text{ } id_A \text{ } id_B \text{ } s \text{ } s' \text{ } s'' \text{ } \#i \text{ } \#j \text{ } \#k . \\
& \text{CreateDynamicState}(\text{sid}, id_A, id_B, s)@i \\
& \wedge \text{CreateDynamicState}(\text{sid}', id_A, id_B, s')@j \\
& \wedge \text{HonestSendStep}(\text{sid}, id_A, id_B, s'')@k \\
& \wedge \#i < \#j \wedge \#j < \#k \Rightarrow \perp.
\end{aligned}$$

Formally, the restriction enforces that only the latest dynamic state can execute  $\text{HonestSendStep}$  actions, i.e., send updates to the peer. We call the resulting model the **Sequential** model.

To model that older sessions can still receive in-flight messages, we copy the rules for receiving messages of the honest parties and add actions that distinguish them from the original rules. Then, we add a restriction that prevents older sessions from using the original receive rules—allowing them to only receive in-flight messages.

**Token-Passing Between Sessions** This policy builds on the *sequential sessions* policy and additionally introduces a secret *token* that is passed from the currently active session to the next one to achieve PCS at the conversation-level. Consequently, there exists only a single dynamic state per user that can create new dynamic states because it is the only one that possesses the token. As we have already shown in our  $\text{Non-Resilient}_{\text{Static}}$  model, this design fails to achieve resilience against dynamic state loss: It is impossible to recover from the loss of this special dynamic state; no other state can create new ones.

**Third-party Providing Token** This policy builds on the previous policy by outsourcing the storage of the token to a trusted third party, which does not lose state. The benefit of this is that the protocol again becomes resistant against dynamic state loss because the affected party can query the third party with their static state to create a new dynamic state.

To model this policy, we add a unique fact,  $\text{TTP}(t, t')$ , for each pair of end-users to our model, which acts as a storage for the current token  $t$  and next token  $t'$ . Additionally, the dynamic state is initialized with the current token. Whenever, a new dynamic state is created, the trusted third party provides the next token only when the current token matches the current dynamic state's token. To recover from dynamic state loss, we also allow for creating a dynamic state from the static state and a token provided by the trusted third party. We call the resulting model  $\text{TokenPassing}$  and refer to our models in [21] for more details.

## 4.4 Formal Analysis Results

In this section, we summarize the results of our formal analysis with TAMARIN.

**1) Non-Resilience** We first show two systems that achieve PCS but are not resilient against state loss, i.e., the loss of *dynamic* and *static* state. In the  $\text{Non-Resilient}_{\text{Dynamic}}$  model, we show the following property for end-user A:

$$\begin{aligned} & \forall \text{sid}_1 \text{ uid}_A \text{ id}_A \text{ uid}_B \text{ id}_B \text{ rk1 rk2 } \#i . \\ & \text{DynamicStateLossA}(\text{sid}_1, \text{uid}_A, \text{id}_A, \text{uid}_B, \text{id}_B, \text{rk1}) @ \#i \\ & \Rightarrow \text{not } (\exists \text{sid}_2 \text{ rk2 } \#j . \#i < \#j \wedge \\ & \text{HonestStepA}(\text{sid}_2, \text{uid}_A, \text{id}_A, \text{uid}_B, \text{rk2}) @ \#j ) \end{aligned}$$

Formally, the property encodes that after the  $\text{DynamicStateLossA}(\text{sid}_1, \text{uid}_A, \text{id}_A, \text{uid}_B, \text{id}_B, \text{rk1})$  action, the user cannot do a communication step  $\text{HonestStepA}(\text{sid}_2, \text{uid}_A, \text{id}_A, \text{uid}_B, \text{rk2})$ . This holds for *any* session that they share with the user  $\text{uid}_B$ . The intuition behind this is straightforward: Losing the *single* dynamic state shared between  $\text{uid}_A$  and  $\text{uid}_B$  without a recovery mechanism locks the user out of any further communication.

In the  $\text{Non-Resilient}_{\text{Static}}$  model, we show that after static state loss the user can no longer participate in the protocol. In TAMARIN, the property is analogous to the previous property, but it reasons about the  $\text{StaticStateLoss}$  fact instead. Since static state is lost, so are all the conversation's concurrent dynamic states, leaving the user locked out of the conversation.

**2) Resilience** In the  $\text{Resilient}_{\text{Dynamic}}$  model, we establish that the users can recover from dynamic state loss and continue communicating with their partner. This is captured by the following property in TAMARIN:

$$\begin{aligned} & \exists \text{sid}_1 \text{sid}_2 \text{uid}_A \text{id}_A \text{uid}_B \text{id}_B \text{rk1} \text{rk2} \#i \#j. \#i < \#j \wedge \\ & \text{DynamicStateLossA}(\text{sid}_1, \text{uid}_A, \text{id}_A, \text{uid}_B, \text{id}_B, \text{rk1}) @ \#i \\ & \wedge \text{HonestStepA}(\text{sid}_2, \text{uid}_A, \text{id}_A, \text{uid}_B, \text{id}_B, \text{rk2}) @ \#j \end{aligned}$$

The property states that it is possible for end-user A to continue executing **HonestStepA** at timepoint  $\#j$ , potentially in a different session, after they previously suffered from a **DynamicStateLossA** event at timepoint  $\#i$  ( $\#i < \#j$ ). Intuitively, this is now possible because the model is no longer restricted to a single session between end-user pairs, i.e., A and B can create a new dynamic state and continue the conversation in a different session.

Analogously, we establish in the  $\text{Resilient}_{\text{Static}}$  model that the static state loss recovery mechanism is also effective. We prove the existence of a trace characterized by the following statement:

$$\begin{aligned} & \exists \text{sid} \text{uid}_A \text{id}_A \text{uid}_B \text{id}_B \text{rk} . \#i < \#j \wedge \\ & \text{StaticStateLoss}(\text{uid}_A, \text{id}_A) @ \#i \wedge \\ & \text{HonestStepA}(\text{sid}, \text{uid}_A, \text{id}_A, \text{uid}_B, \text{id}_B, \text{rk}) @ \#j \end{aligned}$$

Again, the property is analogous to the one for dynamic state loss. Proving this property, we show that a party that suffered a static state loss can continue participating in the protocol afterwards. This property is now achieved for the following technical reasons: Recall from Section 4 that we add a rule raising the **StaticStateRecovery** action to the  $\text{Resilient}_{\text{Static}}$  model, and we change the existing restriction modeling static state loss to be ineffective if the event is raised for a party. As a consequence, static state recovery is now possible.

**3) Impossibility Results** Now, we formally establish our impossibility result:

*In our model, a resilient communication protocol cannot achieve conversation PCS against a full compromise attacker (Definition 6)*

**Definition 6. Conversation PCS - Full** The property states that after the healing of A in session  $\text{sid}_1$ , if there is a protocol step such as send or receive in *any* session, and the attacker knows this secret, this is only possible if they have compromised the victim again after healing or

if they have additionally compromised the partner.

```

 $\forall sid_1 sid_2 uid_A id_A uid_B id_B rk \#i1 \#i2 \#i3 . \#i1 < \#i2 \wedge$ 
// After a party heals session sid1,
HealingA( sid1, uidA, idA, uidB, idB, ... ) @ #i1  $\wedge$ 
// and receives/sends in any session with secret rk,
HonestStep( sid2, uidA, idA, uidB, idB, rk ) @ #i2  $\wedge$ 
// if attacker knows this secret rk,
AttackerKnows( rk ) @ #i3  $\wedge$ 
 $\Rightarrow$  // is because the attacker either:
// compromised the state again after healing,
 $(\exists \#j . \#i1 < \#j \wedge$ 
CompromiseA( uidA, idA, uidB, idB, ... ) @ #j ) )
 $\parallel (\exists \#j . \#i1 < \#j \wedge$  CompromiseDevice( uidA, idA, ) @ #j ) )
// or compromised the state of the partner.
 $\parallel (\exists \#j .$  CompromiseB( uidA, idA, uidB, idB, ... ) @ #j ) )
 $\parallel (\exists \#j .$  CompromiseDevice( uidB, idB, ... ) @ #j ) )

```

To prove our impossibility result, we find counterexamples to Definition 6 in the `ResilientStatic`, the `Sequential`, and the `TokenPassing` model. The impossibility results correspond to the informal intuition built in Section 3. Regardless of how the sessions are managed, they are generated by the static state, which is compromised by the attacker.

**Analysis Times** For all of our models, we specify 14 intermediate helper lemmas to aid TAMARIN in the verification. These lemmas help TAMARIN during its proof search by identifying invariants of the system, e.g., the ordering of specific actions, and allow TAMARIN to discard some traces earlier.

With these lemmas, we are able to fully automatically verify the properties related to the resilience of our *non-resilient* models and our *resilient* models in around 9 seconds per model using TAMARIN’s command-line interface.

To establish our impossibility result, we can find the counterexample in the `ResilientStatic` model automatically with TAMARIN’s CLI in around 330 seconds. Including the time it takes TAMARIN to verify our intermediate lemmas, the verification time for this model is roughly 340 seconds.

Finding the counterexample in both the `Sequential` model and the `TokenPassing` model requires more human interaction. For the former, we use TAMARIN’s GUI and initially guide it by hand, but then we have the tool finish the proof automatically. For the latter, we resort to manually specifying a trace that violates the PCS property and then constructing that trace manually in the GUI. We provide all models and files containing both counterexamples in [21] and Table 2 shows a summary.

## 5 Improved Generalized Session-Handling

From our results in the previous section, we know that conversation PCS cannot be achieved if the attacker compromises the static state (Definition 6). However, messengers can still achieve a stronger PCS than what is currently offered (from Definition 3 to Definition 5). We will first describe the solution we recommend for improving the session-handling layer. Then, we model

| Model                                       | PCS<br>(Definition 6) | Run<br>Time | Helper<br>Lemmas |
|---------------------------------------------|-----------------------|-------------|------------------|
| No Policy<br>(Resilient <sub>Static</sub> ) | ✗                     | 330s        | 14               |
| Sequential                                  | ✗                     | 1h          | 14               |
| TokenPassing                                | ✗                     | 6s*         | 14               |

Table 2: Summary of TAMARIN Impossibility Results.

Our proofs are obtained by using TAMARIN’s *Tactic* feature to guide its proof search, as well as some manual stored proofs (marked by \*). The runtimes consist of automatic verification via TAMARIN’s CLI and automatic verification using its GUI. All models were run on a ThinkPad X1 Carbon Gen 9 with 32Gb of RAM.

the policy in the resilient protocol model and formally prove PCS, thus reducing the attack space in case of a dynamic state compromise.

## 5.1 Proposal

We recommend a generalized policy for all resilient communication systems that aim to improve their post-compromise security guarantees, based on policies from both the literature and deployed in practice.

**Basic Functionalities** The basic functionalities of our solution are based on the Sesame algorithm [30]. Each device locally keeps a list of the partners and their devices, as well as the sessions that they create with each one of those devices.

**Update Policy** Following [20], we also recommend that sessions are created and used sequentially when sending. This means that once a session is replaced, that old session must no longer be used afterwards to encrypt new messages. It should only be used to receive messages and send their acknowledgements.

**Number of Stored Sessions (N)** Each device maintains  $N \geq 2$  sessions: the latest one to send and receive messages, and the previous  $N - 1$  sessions to only receive older messages. To optimize security and usability, we recommend  $N = 2$ . The previous session is needed in two scenarios: a) the session was just updated and there can still be in-flight messages that arrive later, and b) the devices started simultaneous sessions and they need one session to send and the other to receive. In case of simultaneously started sessions, devices could perform an additional check to abandon one of the two, when creation times are relatively close to one another, or they can agree to start a new one. We want to highlight that when an old message is received in one of the previous  $N - 1$  sessions, we recommend to send the acknowledgement of this message *in the old session* in which it was received instead of the latest session. This ensures that Diffie-Hellman shares are exchanged in the old session and it can heal to achieve PCS.

We have consulted with the Signal developers and neither us nor them expect this change to affect the performance or the user-experience of the app. However, this remains to be confirmed in practice, in particular in scenarios that involve high latency or frequent state loss, by future work such as user studies or experiments.

In Section 6.1, we expand on the impact of the parameter  $N$  on security versus resilience.

**Update Policy - User Interface** For user-facing communication systems, the user interface should display received messages in an order that respects the order of the session in which it was sent I.e., a message received in an older session should appear in the correct place in the conversation where the session was active and the message was decrypted. However, this is not how apps currently handle these messages. Instead, they will appear at the bottom (latest part of the conversation) despite being sent in an older session. During normal usage of the messaging app, this behavior occurs rarely; however, if an attacker actively injects messages in the conversation, the end-user is currently not notified of it. To ensure that the end-user is aware of this happening, we additionally recommend highlighting these messages in the user interface.

**Session Deletion** Since the update policy specifies to maintain sequential sessions, the sessions list implements a First-In-First-Out order, i.e., older sessions are removed first. This was already suggested in [30] but not yet implemented in practice in messengers such as Signal. The previous  $N - 1$  sessions can be removed after a predefined time  $T$  and converge to keeping only the current session. The choice of  $T$  specifies the time window during which apps still expect any in-flight out-of-order messages and do not want to re-encrypt them. Choosing a small time window will result in the same issues discussed in Section 6.1.

**Security** Comparing our recommended solution to Sesame, we do not add any additional overhead, while strictly offering a better security guarantee. Adding the update policy of [20] enables a resilient communication protocol to achieve conversation PCS against *dynamic* state compromise<sup>1</sup>, which is otherwise not achieved in a session-management layer without any policies, e.g., Sesame. Our recommendations remove the attack vector of compromising a session and using it to inject *new* messages indefinitely.

However, an attacker that has compromised an old session can still use it to inject *legacy* messages, essentially messages that appear to have been sent in the past. The apps can restrict the attack vector by reducing the number of sessions. Our recommendation to restrict the number of stored sessions to  $N = 2$ , creates a time window in which an attacker can misuse an old compromise, i.e., after  $N$  session updates previously compromised sessions are evicted from the system and can no longer be misused. We show an attack trace in 5.2 to illustrate the subtle security differences that result from the choice of  $N$  in our formal analysis, and we discuss the wider usability-security trade-off in Section 6.1. Essentially, a lower  $N$  has better security (no previous messages are accepted they need to be re-encrypted), while a higher  $N$  offers better resilience (reducing the need for re-encrypting past messages in case of failures).

## 5.2 Formal Analysis

To formally verify our proposed solution, we develop and analyze a TAMARIN model, which we call the **Proposal** model. We build this model starting from the  $\text{Resilient}_{\text{Static}}$  model from Section 4. We add the *sequential session* update policy and a new restriction which models that only the latest two sessions can receive *legacy* messages. This models our recommendation regarding the number of stored sessions ( $N = 2$ ) and update policy. The exact technical details of the TAMARIN model can be found in [21].

**Definition 5. Conversation PCS** We formally modeled conversation PCS against dynamic

---

<sup>1</sup>This does not violate our impossibility result, which involved the compromise of the static state.

state compromise in TAMARIN with the following statement:

```

 $\forall sid_1 sid_2 uid_A id_A uid_B id_B rk \#i1 \#i2 \#i3 . \#i1 < \#i2 \wedge$ 
  // After a party heals session  $sid_1$ ,
  HealingA(  $sid_1, uid_A, id_A, uid_B, id_B, \dots$  ) @  $\#i1 \wedge$ 
  // and receives/sends in any session with secret  $rk$ ,
  HonestStep(  $sid_2, uid_A, id_A, uid_B, id_B, rk$  ) @  $\#i2 \wedge$ 
  // if attacker knows this secret  $rk$ ,
  AttackerKnows(  $rk$  ) @  $\#i3 \wedge$ 
 $\Rightarrow$  // is because the attacker either:
  // compromised the dynamic state again after healing,
   $(\exists \#j . \#i1 < \#j \wedge$ 
    CompromiseA(  $uid_A, id_A, uid_B, id_B, \dots$  ) @  $\#j$  ) )
  // compromised the static state,
   $\parallel (\exists \#j . \text{CompromiseDevice}(uid_A, id_A, ) @ \#j ) )$ 
  // or compromised the state of the partner.
   $\parallel (\exists \#j . \text{CompromiseB}(uid_A, id_A, uid_B, id_B, \dots) @ \#j ) )$ 
   $\parallel (\exists \#j . \text{CompromiseDevice}(uid_B, id_B, \dots) @ \#j ) )$ 

```

For simplicity of presentation, we have omitted some details, e.g., **HealingA** is a shorthand for the role switching that is required to heal A. During role switching A introduces new entropy in their dynamic state. The formal property specifies that after A has healed inside of a session, the attacker can only learn the secret used in an **HonestStep** that occurred in *any* protocol session after the healing if they compromise the dynamic or static state of the victim again after the healing (not protected by PCS) or they compromise the partner.

**Proving Strategy.** To prove the property, we used 14 intermediate lemmas and TAMARIN’s *tactics* feature to prioritize state facts related to the respective user a lemma is concerned with. When proving the PCS property, we prioritized the actions of the attacker, e.g., compromising devices and dynamic states.

We prove the conversation PCS property automatically in TAMARIN’s graphical user interface (GUI) in around 1 hour of time on a ThinkPad X1 Carbon Gen 9 with 32Gb of RAM. Using the GUI was necessary, as we struggled with memory limitations using TAMARIN’s command-line interface. The GUI remedies this by allowing us to *cut* the proof into disjoint case-distinctions which we can then prove separately, needing only a fraction of the memory.

**Reduced Legacy Message Attack Vector.** Finally, we also prove that the **Proposal** model does not contain traces where an end-user receives *legacy* messages from older sessions ( $N > 2$ ). Formally, these traces are characterized by the following statement:

```

 $\exists sid_1 sid_2 sid_3 uid_A id_A uid_B id_B \#i \#j \#k \#l \#m .$ 
 $\#i < \#k \wedge \#k < \#l \wedge$ 
  CreateDynState(  $sid_1, uid_A, id_A, uid_B, id_B, \dots$  ) @  $\#i \wedge$ 
  AttackerSendMsg(  $sid_1, uid_A, id_A, uid_B, id_B, \dots$  ) @  $\#j \wedge$ 
  CreateDynState(  $sid_2, uid_A, id_A, uid_B, id_B, \dots$  ) @  $\#k \wedge$ 
  CreateDynState(  $sid_3, uid_A, id_A, uid_B, id_B, \dots$  ) @  $\#l \wedge$ 
  ReceiveLegacyMsg(  $sid_1, uid_A, id_A, uid_B, id_B, \dots$  ) @  $\#m$ 

```

The property encodes the existence of a particular attack, and states that the following is possible: If two new sessions are created,  $sid_2$  at timepoint  $\#k$ , and  $sid_3$  at timepoint  $\#l$ , then the

user can still accept messages sent by an attacker from a previous session *sid1*. For the model with our policy restrictions, TAMARIN automatically disproves this property almost instantly (5s), i.e., it proves that no such trace exists. In contrast, when removing the  $N = 2$  policy restriction from the model, we can prove the property true and display a corresponding attack trace in the GUI in 5s. The absence of these traces in the Proposal model shows that our proposal reduces the attack surface.

## 6 Discussion

### 6.1 The design space of the number of stored sessions: Demystifying Signal’s 40 sessions

One interesting question is related to the number of sessions the systems need to store and their implications for resilience and security.

Consider a system that immediately discards the older session when creating a new one,  $N = 1$ . The following scenario would take place: A and B independently start new sessions at the same time, and upon receiving the message from their partner, they discard the local session and accept the incoming one. Now, A sends messages using the session started by B,  $D_{0,0}^B$ ; however, B has deleted that session and cannot decrypt. All the messages received will trigger a retry request to be re-encrypted in a new session. Figure 3 depicts the described scenario. The same logic applies when A, respectively B, refreshes their session and all in-flight messages from the older session trigger a retry request.

Since A and B communicate asynchronously, it is possible that messages keep crossing. In addition, if A loses the dynamic state  $M$  times, a message sent in a session before the loss,  $N_{\text{sender}} < N_{\text{current}} - M$ , will trigger a retry request upon being received. When apps evaluate that end-users refresh their sessions very frequently, e.g., due to faulty hardware, and do not want the retransmitting overhead, the number of stored sessions can be modified to their needs,  $N = M$ . However, storing a large  $N$  leads to the attacker being able to misuse any of those sessions to inject messages as shown in previous works [20, 42]. The parameter  $N$  can be seen as a trade-off between the resilience against an unreliable environment and the security against the attack vector of using older sessions to circumvent the healing of a specific session. The Signal messenger (current version: Android app v7.24.2, November 2024, and any previous versions) stores 40 session at one time. Following our previous argument, the number of sessions equals the window of tolerance of the app to receive out-of-order messages from previous sessions and decrypt without retransmitting messages. Previously, Signal (Android app v5.40.4, May 2022) restarted sessions every hour, which translates to accepting messages without retransmitting from the last 40 hours or the last 40 active sessions. Any other messages would trigger a retry request and therefore a new session. The current Signal version does no longer refresh its sessions every hour and, as a result, it accepts messages sent in the latest 40 active sessions.

### 6.2 Towards compromise detection

From our results that PCS cannot be achieved at the conversation layer, it becomes clear that the problem repeats itself in detecting the compromise. The intuition behind this is similar in that allowed operations due to failure modes enable the attacker to mimic these behaviors and become indistinguishable from the victim. Any deployable detection mechanisms would need to rely on an out-of-band channel or require the explicit end-user’s involvement to detect at least a *subset* of the more trivial attacks.

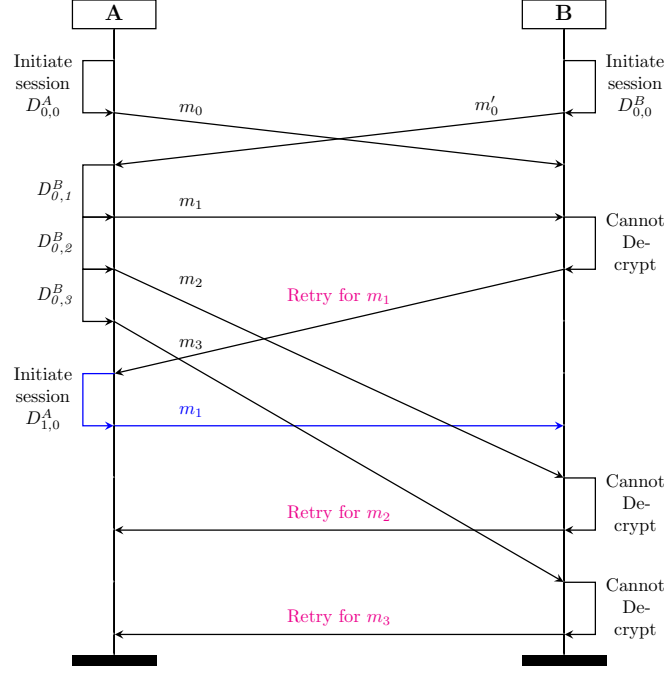


Figure 3: **Round-trips overhead on storing  $N = 1$  session.** A and B storing a single session triggers retry requests on out-of-order messages from the previous session. If parties store  $N = 2$  sessions, they are decrypted upon being received.

We can already observe this in the literature; however, substantial work is needed in the future to formally prove the claim. Principles and foundations for compromise detection were proposed in [34], but this work did not consider potential failure modes such as state loss. More recently, [8] develops a detection mechanism against an active attacker using ordinals (the epoch and message numbers of the Double Ratchet) and also a hash of the message transcript. Under the realistic assumptions of real-world implementations, their construction yields *false positives*, i.e., an attacker will be flagged during normal operations of the app. Consider the simple example where A restores an old backup, starts a new session with partner B and sends a message. This will incorrectly result in a compromise detection, since the ordinals and the hashed transcript do not match. Additionally, if the scheme is deployed strictly per session, the attacker can circumvent it by spawning a new session, while remaining undetected on the session between A and B. Similarly in [18, 20, 34], the mechanisms either do not consider multi-sessions or desynchronization. If these mechanisms are used with multiple sessions or in the presence of failure modes, this typically either introduces false positives or possibilities for circumvention, thus rendering them ineffective in practice.

## 7 Conclusions

We have shown that real-world communication systems cannot simultaneously provide full conversation PCS and resilience against specific forms of state loss, such as loss of dynamic state. However, these forms of state loss are unavoidable in real-world deployment for a variety of reasons, including memory glitches or OS-level backups. For designs like Signal and Matrix that use concurrent sessions, we have shown a trade-off between resilience and security.

It is tempting to think that rotating long-term keys can circumvent these problems, but such solutions essentially suffer from the exact same impossibility result under slightly modified conditions on state loss. In the end, it seems that achieving stronger PCS guarantees requires, or more likely needs to assume, that state loss occurs extremely infrequently, in which case one might be able to rely on external assumptions such as parallel out-of-band (OOB) solutions. However, most real-world communication systems cannot realistically assume that OOB mechanisms can be arbitrarily and immediately invoked, thereby leading to a trade-off with availability (and in case of human end-users, usability).

If full PCS cannot be achieved, another option is to focus instead on compromise detection. While we argued in Section 6.2 that detection suffers from similar problems, it may still be possible to achieve in some cases. However, before designers consider this direction, they must first consider the question: If a potential compromise were to be detected, what would be the possible consequences, given that it is usually impossible to differentiate the honest party from the adversary? In high-end security settings or with high-risk users, aborting communications entirely and relying on OOB mechanisms might be possible. However, where high availability or usability is the priority, this may not be a realistic option.

## References

- [1] Martin R. Albrecht, Benjamin Dowling, and Daniel Jones. Device-Oriented Group Messaging: A Formal Cryptographic Analysis of Matrix’ Core. In *SP*, pages 2666–1685. IEEE, 2024.
- [2] Joël Alwen, Benedikt Auerbach, Miguel Cueto Noval, Karen Klein, Guillermo Pascual-Perez, and Krzysztof Pietrzak. DeCAF: Decentralizable CGKA with Fast Healing. In *SCN (2)*, volume 14974 of *Lecture Notes in Computer Science*, pages 294–313. Springer, 2024.
- [3] Joël Alwen, Sandro Coretti, and Yevgeniy Dodis. The Double Ratchet: Security Notions, Proofs, and Modularization for the Signal Protocol. In *EUROCRYPT (1)*, volume 11476 of *Lecture Notes in Computer Science*, pages 129–158. Springer, 2019.
- [4] Joël Alwen, Sandro Coretti, Daniel Jost, and Marta Mularczyk. Continuous Group Key Agreement with Active Security. In *TCC (2)*, volume 12551 of *Lecture Notes in Computer Science*, pages 261–290. Springer, 2020.
- [5] David Balbás, Daniel Collins, and Phillip Gajland. WhatsApp with Sender Keys? Analysis, Improvements and Security Proofs. 14442:307–341, 2023.
- [6] David Balbás, Daniel Collins, and Serge Vaudenay. Cryptographic Administration for Secure Group Messaging. In *USENIX Security Symposium*, pages 1253–1270. USENIX Association, 2023.
- [7] Richard Barnes, Benjamin Beurdouche, Raphael Robert, Jon Millican, Emad Omara, and Katriel Cohn-Gordon. The Messaging Layer Security (MLS) Protocol, 2023.

- [8] Khashayar Barooti, Daniel Collins, Simone Colombo, Loïs Huguenin-Dumittan, and Serge Vaudenay. On active attack detection in messaging with immediate decryption. In *CRYPTO (4)*, volume 14084 of *Lecture Notes in Computer Science*, pages 362–395. Springer, 2023.
- [9] Mihir Bellare, Asha Camper Singh, Joseph Jaeger, Maya Nyayapati, and Igors Stepanovs. Ratcheted Encryption and Key Exchange: The Security of Messaging. In *CRYPTO (3)*, volume 10403 of *Lecture Notes in Computer Science*, pages 619–650. Springer, 2017.
- [10] Alexander Bienstock, Yevgeniy Dodis, and Paul Rösler. On the Price of Concurrency in Group Ratcheting Protocols. In *TCC (2)*, volume 12551 of *Lecture Notes in Computer Science*, pages 198–228. Springer, 2020.
- [11] Alexander Bienstock, Jaiden Fairoze, Sanjam Garg, Pratyay Mukherjee, and Srinivasan Raghuraman. A More Complete Analysis of the Signal Double Ratchet Algorithm. In *CRYPTO (1)*, volume 13507 of *Lecture Notes in Computer Science*, pages 784–813. Springer, 2022.
- [12] Olivier Blazy, Angèle Bossuat, Xavier Bultel, Pierre-Alain Fouque, Cristina Onete, and Elena Pagnin. SAID: Reshaping Signal into an Identity-Based Asynchronous Messaging Protocol with Authenticated Ratcheting. In *EuroS&P*, pages 294–309. IEEE, 2019.
- [13] Olivier Blazy, Ioana Boureanu, Pascal Lafourcade, Cristina Onete, and Léo Robert. How fast do you heal? A taxonomy for post-compromise security in secure-channel establishment. In *USENIX Security Symposium*, pages 5917–5934. USENIX Association, 2023.
- [14] Sébastien Champion, Julien Devigne, Céline Duguey, and Pierre-Alain Fouque. Multi-Device for Signal. In *ACNS (2)*, volume 12147 of *Lecture Notes in Computer Science*, pages 167–187. Springer, 2020.
- [15] Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A Formal Security Analysis of the Signal Messaging Protocol. *J. Cryptol.*, 33(4):1914–1983, 2020.
- [16] Katriel Cohn-Gordon, Cas Cremers, and Luke Garratt. On Post-compromise Security. In *CSF*, pages 164–178. IEEE Computer Society, 2016.
- [17] Katriel Cohn-Gordon, Cas Cremers, Luke Garratt, Jon Millican, and Kevin Milner. On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guarantees. In *CCS*, pages 1802–1819. ACM, 2018.
- [18] Cas Cremers, Jaiden Fairoze, Benjamin Kiesl, and Aurora Naska. Clone Detection in Secure Messaging: Improving Post-Compromise Security in Practice. In *CCS*, pages 1481–1495. ACM, 2020.
- [19] Cas Cremers, Britta Hale, and Konrad Kohbrok. The Complexities of Healing in Secure Group Messaging: Why Cross-Group Effects Matter. In *USENIX Security Symposium*, pages 1847–1864. USENIX Association, 2021.
- [20] Cas Cremers, Charlie Jacomme, and Aurora Naska. Formal Analysis of Session-Handling in Secure Messaging: Lifting Security from Sessions to Conversations. In *USENIX Security Symposium*, pages 1235–1252. USENIX Association, 2023.
- [21] Cas Cremers, Niklas Medinger, and Aurora Naska. PCS Analysis Models. <https://github.com/pcsanalysis/seprint/pcsanalysis/seprint>, November 2024.

- [22] Benjamin Dowling and Britta Hale. Authenticated Continuous Key Agreement: Active MitM Detection and Prevention. 2023. URL: <https://eprint.iacr.org/2023/228>.
- [23] F. Betül Durak and Serge Vaudenay. Bidirectional Asynchronous Ratcheted Key Agreement with Linear Complexity. 2018. URL: <https://eprint.iacr.org/2018/889>, doi:10.1007/978-3-030-26834-3\_20.
- [24] Google Messages. Technical Paper. online, Accessed: 26.09.2024. URL: [https://www.gstatic.com/messages/papers/messages\\_e2ee.pdf](https://www.gstatic.com/messages/papers/messages_e2ee.pdf).
- [25] Joseph Jaeger and Igors Stepanovs. Optimal Channel Security Against Fine-Grained State Compromise: The Safety of Messaging. In *CRYPTO (1)*, volume 10991 of *Lecture Notes in Computer Science*, pages 33–62. Springer, 2018.
- [26] Daniel Jost, Ueli Maurer, and Marta Mularczyk. Efficient Ratcheting: Almost-Optimal Guarantees for Secure Messaging. In *EUROCRYPT (1)*, volume 11476 of *Lecture Notes in Computer Science*, pages 159–188. Springer, 2019.
- [27] Julia Len, Esha Ghosh, Paul Grubbs, and Paul Rösler. Interoperability in End-to-End Encrypted Messaging. 2023. URL: <https://eprint.iacr.org/2023/386>.
- [28] Xin Li, Michael C. Huang, Kai Shen, and Lingkun Chu. A Realistic Evaluation of Memory Hardware Errors and Software System Susceptibility. In *USENIX ATC*. USENIX Association, 2010.
- [29] Felix Linker, Ralf Sasse, and David Basin. A Formal Analysis of Apple’s iMessage PQ3 Protocol. Cryptology ePrint Archive, Paper 2024/1395, 2024. URL: <https://eprint.iacr.org/2024/1395>.
- [30] Moxie Marlinspike and Trevor Perrin. The Sesame Algorithm: Session Management for Asynchronous Message Encryption. 2017. URL: <https://signal.org/docs/specifications/sesame/>.
- [31] Moxie Marlinspike and Trevor Perrin. The X3DH Key Agreement Protocol. Accessed: 26.09.2024. URL: <https://signal.org/docs/specifications/x3dh/>.
- [32] Marlinspike, Moxie. Simplifying OTR deniability. online, Accessed: 26.09.2024. URL: <https://signal.org/blog/simplifying-otr-deniability/>.
- [33] Simon Meier, Benedikt Schmidt, Cas Cremers, and David A. Basin. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *CAV*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701. Springer, 2013.
- [34] Kevin Milner, Cas Cremers, Jiangshan Yu, and Mark Ryan. Automatically Detecting the Misuse of Secrets: Foundations, Design Principles, and Applications. In *CSF*, pages 203–216. IEEE Computer Society, 2017.
- [35] Trevor Perrin and Moxie Marlinspike. The Double Ratchet Algorithm. Accessed: 26.09.2024. URL: <https://signal.org/docs/specifications/doubleratchet/>.
- [36] Paul Rösler, Christian Mainka, and Jörg Schwenk. More is Less: On the End-to-End Security of Group Chats in Signal, WhatsApp, and Threema. In *EuroSCP*, pages 415–429. IEEE, 2018.

- [37] Signal Messenger. Technical Specification. online, Accessed: 26.09.2024. URL: <https://signal.org/docs/>.
- [38] Douglas Stebila. Security analysis of the iMessage PQ3 protocol. Cryptology ePrint Archive, Paper 2024/357, 2024. URL: <https://eprint.iacr.org/2024/357>.
- [39] The Matrix.org Foundation. Matrix specification: Client-Server API. online, Accessed: 26.09.2024. URL: <https://spec.matrix.org/v1.11/client-server-api/>.
- [40] The Matrix.org Foundation. Megolm. online, Accessed: 26.09.2024. URL: <https://gitlab.matrix.org/matrix-org/olm/-/raw/master/docs/megolm.md>.
- [41] The Matrix.org Foundation. Olm. online, Accessed: 26.09.2024. URL: <https://gitlab.matrix.org/matrix-org/olm/-/raw/master/docs/olm.md>.
- [42] Andreas Tsouloupas. Breaking Cryptography in the Wild: The Loose Ends of the Wire. Master's thesis, ETH Zurich, 2023. URL: <https://doi.org/10.3929/ethz-b-000673362>.
- [43] WhatsApp. Technical White Paper. online, Accessed: 26.09.2024. URL: <https://www.whatsapp.com/security/>.
- [44] Wire. Proteus. online, Accessed: 26.09.2024. URL: <https://github.com/wireapp/proteus>.

## A Changelog

- Version 1.0, November 22, 2024.
- Version 1.0.1, November 25, 2024:
  - Fixed typos.
  - Added Changelog.